

TODO: talk about OFAC regulated relays (mevwatch.info)

1 Introduction

2 Maximal Extractable Value

It is worth making a digression about how the orders are submitted and validated on the Ethereum blockchain. Indeed, the peculiar framework employed allows the agents charged with handling orders to extract profit from transaction manipulations through operations like rearrangements, additions, or removals. These profits add on top of the standard rewards and transaction fees. In general, this kind of gain is known as Maximal Extractable Value (MEV)¹, and it is one of the core dynamics present in DEXs and, so, in Uniswap v3.

2.1 Proposer-Builder Separation (PBS)

In TradFi, orders are processed instantaneously, and the time difference between submission from the user and insertion in the limit order book is usually of the order of milliseconds. Therefore, it is quite impossible to find trades sharing the same exact timestamp. On the other hand, the order flow in DEXs is lumpy, as transactions are grouped into blocks that are validated sparsely in time, currently, on the Ethereum blockchain, every 12 seconds. This means that the orders submitted are not immediately active in the market, but instead need to wait before being added to the blockchain. The orders submitted between the validation of two blocks are received by an Ethereum node and stored in the so-called memory pool (*mempool*). We can think of the mempool as a 'waiting room' where transactions stay before being validated and appended to the next block. The (public) mempool is the best field in which MEV proliferates. At a high level, the mechanism of MEV extraction is quite simple: since all the information about the pending transaction is accessible to everyone², in the period between the validation of two adjacent blocks, all users have the possibility to add their own transaction to the mempool and exploit the effects of what the currently pending transactions will have (that can be simulated deterministically). Before 2022, traders fighting for MEV, that we will call from now on *MEV searchers*, kept offering higher and higher priority fees³ so that the miner or validator would place their transaction first. Ordinary users got caught in the cross-fire and had to pay much higher fees for any kind of operation, especially during events prone to high MEV rents. For example, an MEV searcher that wants to front run a large swap, should have attached to the front run swap an higher priority fees than the victim swap to rank before it.

Around 2022, in the Ethereum community a new concept started to be discussed [?]: the Proposer-Builder-Separation (PBS), a mechanism aimed to distribute the role of creating new blocks among different actors with the aim of reducing centralization, censorship and the negative effects of the priority gas fee wars just described⁴. PBS changes who constructs a block by separating the builder role (execution + ordering) from the proposer role (consensus). The validator still runs the execution engine to verify the final payload before signing. The former is put in the hand of the *block builders*, the latter operated by the *proposer*, i.e. the validators. Currently, PBS is implemented by third party services, and so it is not an active part of the consensus protocol on Ethereum, but the Ethereum research and client development communities are working towards a protocol-level implementation of PBS, which is expected to be included in a future hard fork [?].

The most adopted software implementation of the PBS is MEV-Boost by Flashbots (<https://boost.flashbots.net/>). Almost 90% of the blocks are currently validated using MEV-Boost [?]. Again, the general idea is to distribute the role of creating block to a new actor, the *block builder*, and

¹Historically, it has been introduced as Miner Extractable Value, see [?].

²For example, via Etherscan <https://etherscan.io/mempool/all>

³The priority fee is a tip that the users add to their transactions to gain priority in the mempool. It is one of the two components of the transactions fees (known as gas fees). The other one is the base fee, chosen deterministically based on the previous blocks congestion. It is burned, so no one gain it.

⁴technically, they are called Priority-Gas Auctions (PGAs)

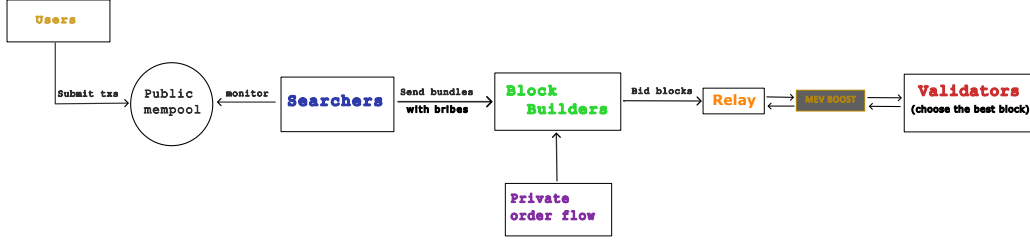


Figure 1: PBS schema

delegate just the validation of the block to the validator. The block builder receives (atomic⁵) bundles of transactions constructed by the MEV searchers from the mempool in a bid-auction mechanism. The searchers therefore must attach a bribe to the bundles and the auction is handled off-chain. Then, builders construct block proposals that are submitted to a validator, again in a bid-auction mechanism which is managed by trusted third-parties known as *relays*. Relays act as neutral brokers: builders stream just the block headers plus bids to the relays, the validator asks each relay for its best header and signs the richest one. Finally the relay reveals the full block only after it sees the validator's signature that guarantee a fair exchange between the two parties, preventing the validator from stealing the builder's work and the builder to under-bid the validator. Figure 1 illustrates this chain of actions. For a more detailed description of this chain of actors, see [?]. Therefore, huge gas-fee spikes are now less common because the bidding happens off-chain, between builders, not in the public mempool, see Figure 2. On the other hand, even if one of the goals was to reduce the centralized power that validators had, PBS moved this power to an even smaller set of professional builders that handle most blocks, so the system is more orderly but also more centralized than before. Figure 3, taken from [?], shows the flows of the block proposal from the block builders to the validators, through the relays. Additionally, there is also the risk of a relay failure, as already happened [?] [?].

However, this is not the entire story, as in the last couple of years high-value wallets and bots have increasingly bypassed the public mempool, sending bundles straight to builders via 'private RPC' channels. This exclusive order flow (EOF) gives the builder a first (often sole) look at the trade, letting them avoid public priority gas fee auctions, pair the transaction with an optimal back-run⁶, and quote a bigger bid to the validator, increasing the chance of winning the next block. Depending on the agreement, part of the extracted MEV may be rebated to the EOF sender, or the sender may pay a flat coin-base tip for protection against MEV.

Summarizing, we have three main actors in PBS: MEV-searchers, Block Builders and Validators. Their profits can be written as

$$\textbf{Searcher: } \pi_s = M - (C_{\text{gas}} + D_{\text{sb}} + C_{s,\text{infra}}) \quad (1)$$

$$\textbf{Builder: } \pi_b = R - (1 + \rho)B - R_{\text{refund}} - C_{b,\text{infra}} \quad (2)$$

$$\textbf{Validator: } \pi_v = B + R_{\text{consensus}} - C_{v,\text{infra}} \quad (3)$$

$$R_{\text{refund}} = \varphi D_{\text{sb}}, \quad 0 \leq \varphi \leq 1.$$

$$R = \sum_{i \in \text{tx}} (\text{priority fee}_i + \text{direct transfer}_i).$$

⁵An atomic transaction is either fully executed or totally reverted, depending whether its actions are feasible.

⁶Usually, there are agreements between the order flow provider and the block builder that allows the latter to just front run the former

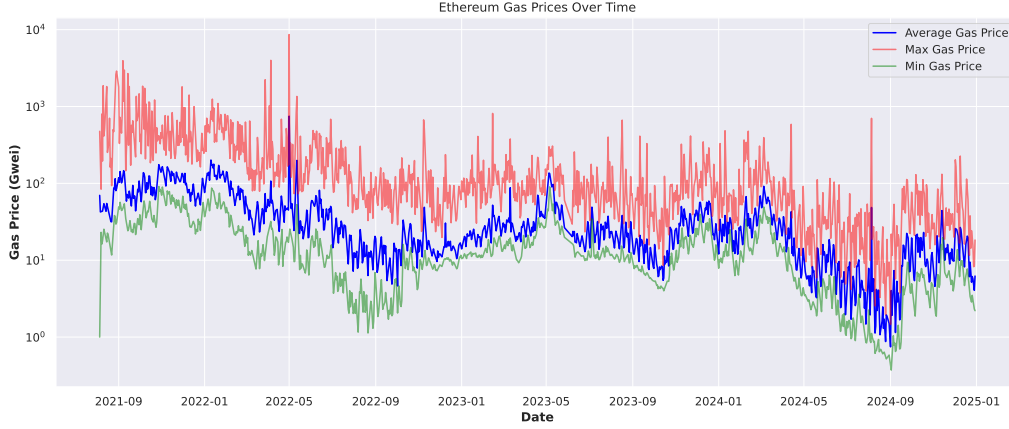


Figure 2: Hiastorical gas price from 05/08/2021 (first available day of the DexGuru Ethereum Gas Tracker) to 31/12/2024. The Min Gas Price represents the lowest fee paid for a transaction to be included in a block, while the Max Gas Price is the highest fee paid in that same block — reflecting the range of urgency and cost users were willing to pay.

Symbol	Meaning
R	Raw block value (priority-fee tips + direct transfers)
B	Builder $\rightarrow$ proposer payment (coinbase transfer)
M	MEV revenue realised in the searcher’s bundle(s)
C_{gas}	Gas actually paid by the searcher’s bundle
D_{sb}	Direct ETH transfer (tip) from searcher to builder
φ	Refund ratio (0–1) set by MEV-Share or similar
R_{refund}	On-chain rebate to user: φD_{sb}
ρ	Relay-fee fraction on the builder’s bid (0–1)
$C_{s,\text{infra}}$	Off-chain infrastructure costs of the searcher
$C_{b,\text{infra}}$	Builder infrastructure & order-flow costs
$C_{v,\text{infra}}$	Validator infrastructure costs
$R_{\text{consensus}}$	Protocol-level block-proposal reward

2.2 MEV/Manipulation Strategies

In this section we present a narrative that walks through different MEV strategies and carefully draws parallels to its closest equivalent in traditional finance (TradFi), unpacking how each tactic works, what the motivations are, and where the two domains differ due to rules, infrastructure, or ethics. We will focus on the most widely used strategies ⁷. They include arbitrages, sandwich attacks and Just-in-Time liquidity.

2.2.1 Flash-loan attacks

A flash loan allows any Ethereum user to borrow an arbitrary amount of ERC-20 liquidity without any kind of collateral provided that the entire borrow $\rightarrow$ exploit $\rightarrow$ repay sequence is executed atomically inside a single transaction. This means that the lender’s solvency risk is zero (the call reverts if the repayment fails). All the strategies described below can (and actually do) include a flash loan as initial step. Indeed, a meta-study of the 100 largest DeFi hacks through 2024 finds that roughly 60% of all price-manipulation exploits relied on flash loans [?]. In addition, flash loans can also be used to perform hijack on governance-tokens based protocols, like Decentralized Autonomous Organizations (DAOs). In this case, the attacker flash-borrow $>51\%$ of a token supply, pass a malicious proposal in the same transaction, drain the treasury, and repay.

⁷For a comprehensive survey about MEV in DeFi, see [?].

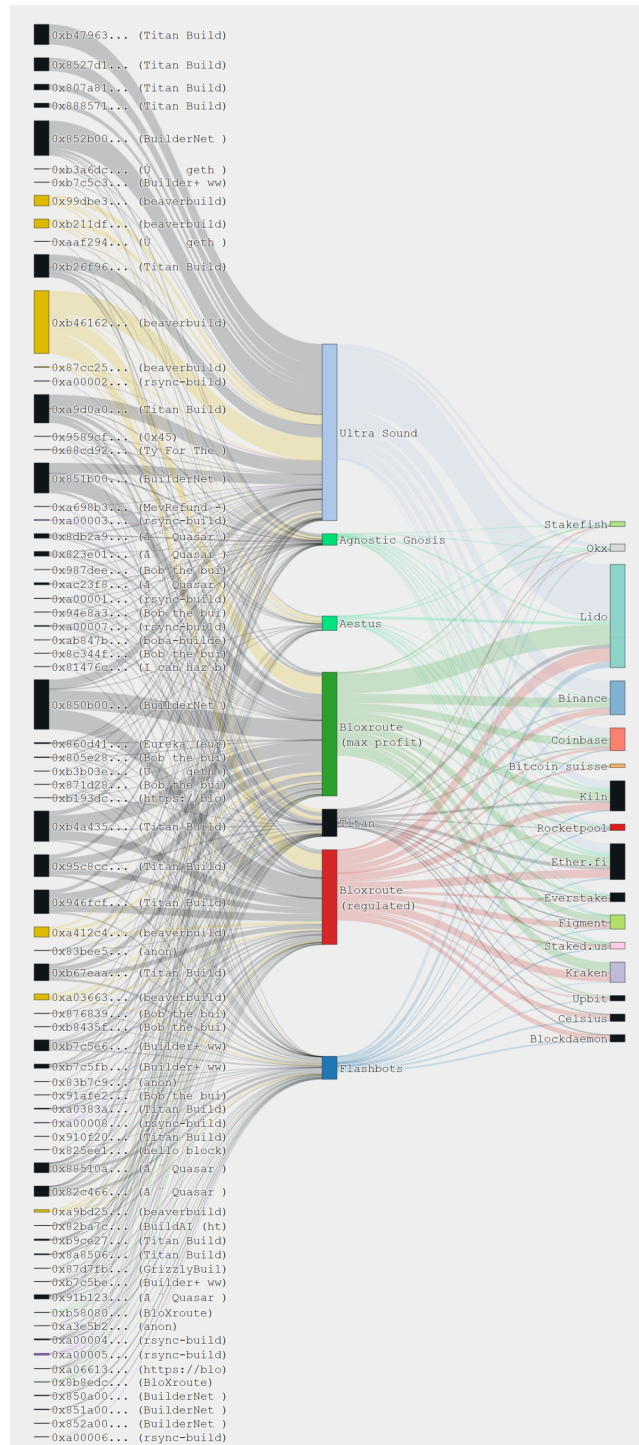


Figure 3: Caption

There is no analogue of unlimited, unsecured and atomic credit in TradFi.

2.2.2 Arbitrage

In these cases, MEV searchers can exploit price mismatches between two pools, two DEXs or even DEX and CEX. In such a way, they contribute to market efficiency. We can distinguish between atomic arbitrages and non-atomic arbitrages. The first ones involve only on-chain operations and are executed using atomic transactions to buy the cheaper asset and simultaneously sell the more expensive one, usually within a single block. Furthermore, thanks to the atomicity of smart contracts, the searcher can also take advantage of flash loans⁸ to perform the strategy without upfront capital. It is therefore an almost riskless strategy since if the transaction fails, the searcher only loses the gas fee paid. Instead, non-atomic arbitrages refer to DEX-CEX arbitrages, where the searcher must interact with an off-chain platform (the CEX) and so she cannot exploit the atomicity of the smart contracts, exposing herself to the risk of a non-profitable arbitrage due to adverse price movements before the validation of her transactions.

What we have just described can be labeled as beneficial for the market, since it helps the information flows through the different market venues in the DeFi ecosystem. However, it is important to note that there are some malicious types of arbitrage that can be called 'price manipulation arbitrages'. In this scenario, a user takes a flash loan, uses part of it to borrow a token (like WBTC) on a lending platform, and another part to execute a leveraged trade that pushes up the token's price on a low-liquidity AMM. They then sell the borrowed tokens at the inflated price and repay the flash loan, keeping the profit. All of this is possible due to smart contract bugs being exploited.

In order to understand this dynamics, it can be helpful to illustrate one famous example of this attack. It happened in 2020 on the protocol bZx. In that case, the attacker did exactly what we have already discussed: he took a flash loan of 10,000 ETH and used 5,500 ETH as collateral on Compound to borrow 112 WBTC, while using 1,300 ETH to open a 5x leveraged margin trade on bZx. The margin trade routed through Kyber and executed a large ETH-to-WBTC swap on Uniswap, which had low liquidity at the time. This trade itself caused the price of WBTC to spike from around 38 ETH to over 109 ETH. Due to a bug in the bZx smart contract, the system failed to perform a proper collateralization check after the trade and accepted the manipulated price as valid. As a result, the attacker was able to sell the 112 borrowed WBTC for 6,871 ETH, repay the 10,000 ETH flash loan, and walk away with a net profit of around 1,271 ETH, worth approximately \$355,000 at the time, leaving the protocol with bad debt.

In traditional finance, this maps closely to multi-venue arbitrage, a classic strategy in equity and FX markets. High-frequency trading (HFT) firms monitor multiple exchanges using direct, ultra-low-latency data feeds. When they detect a stale price on one venue they race to buy low and sell high before the prices converge. Similar strategies are used between ETFs and their underlying baskets.

Key differences:

- In TradFi, trades are not atomic—there's real execution and inventory risk during the latency window.
- in TradFi infrastructure is a huge barrier: collocation, proprietary data feeds, and microwave networks can cost tens of millions annually.
- In DeFi, a simple Ethereum transaction with the right logic can do the same work for a few dollars in gas.

2.2.3 Sandwich attacks

Sandwich attacks are strategies designed for gaining from the price impact of a large swap (performed by the victim) pending in the mempool. The MEV searcher acts as follows. Firstly, a swap is placed

⁸A flash loan on Ethereum is a type of unsecured loan offered by certain decentralized finance (DeFi) protocols (e.g., Aave or dYdX) that allows users to borrow assets instantly and without collateral, as long as the loan is repaid within the same blockchain transaction

in the same direction as the victim (also known as front run); then, the victim swap is executed ⁹; lastly, the attacker closes the position by placing a swap with the same amount and opposite sign to the front run (also known as back run). This type of attack harms market participants, as the victim pays a higher price due to the impact of the front run swap.

The victim can protect herself from this type of attack by setting a low slippage tolerance on her transaction. This means that if the swap would result in receiving less than a certain minimum amount, the transaction will automatically fail. As a result, the attacker must carefully choose how much to front-run—if they push the price too far, the victim’s transaction will not go through, and the attack fails.

The TradFi analogue is front-running, either in its classic form (e.g., a broker trading ahead of a client order) or in more modern, legal gray zones like latency arbitrage. HFT firms use predictive models and "ping" orders to infer large institutional activity. If detected early, HFTs may quickly buy shares in one venue before the order hits and sell them at a slightly higher price moments later.

Key differences:

- On Italian markets, dealing on non-public client information is prohibited by the EU Market Abuse Regulation (MAR) and Articles 184-187-ter of Italy’s Consolidated Finance Act (TUF), so front-running is a criminal behaviour.
- In DeFi, the mempool is public. This makes sandwich attacks legally murky: technically, the bot is acting on public data, not insider information.
- TradFi has surveillance systems, anti-internalization rules, and dark pool disclosures. DeFi is experimenting with privacy-preserving alternatives (e.g., encrypted mempools, SUAVE) but lacks formal enforcement.

2.2.4 Just-in-Time (JIT) liquidity

JIT liquidity is a strategy for gaining the fee that a large swap in the mempool will pay once executed. The MEV searcher deposits liquidity in a very narrow price range (containing the current price) just before the transaction is processed, earning most of the trading fee, and removes the liquidity immediately after the transaction. This strategy is also referred to as *LP sandwich* or *active liquidity provision*¹⁰ [?], as the liquidity provision is elicited by a pending order in the mempool. The liquidity provider never intends to provide ongoing liquidity; they simply "ride" the trade. In contrast to sandwich attacks, JIT liquidity is beneficial for the end user due to the enhanced liquidity that lower the price impact of the swap.

In TradFi there is not a direct analogue of this strategy, but it resembles fleeting quotes and rebate-seeking behavior. On venues available to Italian firms, HFTs sometimes flash tight quotes on pan-European MTFs such as Cboe Europe CXE/BXE, which still rebate about 0.15 bps to passive fills. By contrast the main Euronext Milan order book merely waives or halves the passive fee rather than paying cash.

Key differences:

- TradFi exchanges now limit these tactics. MiFID II empowers CONSOB and Borsa Italiana to impose order-to-trade-ratio caps and minimum quote-life parameters, preventing excessive 'quote flicker' by HFT algorithms.
- In Uniswap, there is no time-in-force constraint—adding and removing liquidity can happen in a single atomic transaction.
- TradFi rebates are paid by the venue; in DeFi, traders themselves pay LP fees. The design incentive is inverted.

⁹There is of course the possibility of multiple victims' swaps

¹⁰Despite this definition is very appealing, it should not be confused with the so-called *active liquidity*, that is the sum of all the liquidity whose range contains the price.

2.2.5 Mixed (Sandwich+JIT) attacks

It is worth noting that the state-of-the-art to exploit a large swap in DeFi is a combination of JIT and Sandwich attacks. When a large swap is detected in the mempool, a mixing attack made up of a JIT encapsulated into a sandwich could sometimes bring the highest profit. Specifically, the exact scheme is: front run, mint liquidity, victim swap, burn liquidity, back run.

MEV strategies aren't new—they are just smart-contract-powered expressions of long-standing financial instincts: arbitrage, front-running, fleeting liquidity provisioning etc. What sets DeFi apart is its:

- **Atomicity:** execution is all-or-nothing in a single block
- **Transparency:** everything is visible (and exploitable) in the mempool.
- **Permissionlessness:** anyone with code and gas can participate—no licenses, no seat fees.

Meanwhile, TradFi imposes structural constraints through surveillance, regulatory rules, and operational friction (e.g., latency, risk desks). But at their core, both systems reward the same thing: speed, insight, and the ability to act before others do.

3 Which strategy?

The main choice that an MEV searcher is faced to understand what is the best strategy to employ according to the size of the victim's swap. In this section, we derive the profits of four different strategies:

1. Just-in-Time (JIT) liquidity
2. Back run arbitrage
3. Sandwich Attack
4. Mixed strategy JIT+Sandwich Attack

These profits, minus the gas fees paid to perform them, correspond to the maximum bribe that the MEV searcher can pay to the block builder while guaranteeing a positive profit. Table 1 summarizes the notation used in this section

Throughout this section we adopt the standard Uniswap v3 single-tick equivalence: let $S = \sqrt{P} = \sqrt{Y/X}$ be the square-root price and L the active liquidity. Define the virtual reserves $\tilde{x} = L/S$ and $\tilde{y} = LS$. Within a tick, swaps update $(\tilde{x}, \tilde{y})$ exactly as a constant-product AMM, and an ultra-narrow JIT mint is a liquidity bump $L \mapsto (1 + \alpha)L$ that keeps the price fixed. All formulas in this section hold if x_i, y_i are interpreted as $\tilde{x}_i, \tilde{y}_i$ and no tick boundary is crossed during any leg (front-run, victim, burn, back-run). A tick-aware generalization rule (multi-tick traversal with updates to L) is provided in Appendix A.

The analysis values every term in token X using the pre-strategy marginal price $P_0 = y_0/x_0$.

We use the term "victim" to refer to the user whose swap is included in the bundle. However, this label can be misleading in certain contexts. In JIT liquidity provision, the strategy often benefits the user by reducing price impact through enhanced liquidity. Similarly, in back-run arbitrage, the user's transaction remains unaffected, as the arbitrage occurs after their swap and does not inflict any harm.

Symbol	Meaning
f	Uniswap fee rate (e.g., 0.0005, 0.003, 0.01)
S	Size of the victim's swap (in token X)
σ	Normalized victim's swap size, $S = \sigma x_0$
r	Net input rate after fee, $r = 1 - f$
s	Attacker front-run input (in token X)
ε	Normalized front-run size, $s = \varepsilon x_0$
α	Multiplier of the reserves chosen by the searcher for LP
x_0, y_0	Pool reserves before the beginning of the strategy
P_0	Marginal-price of X in terms of Y before the beginning of the strategy, $P_0 = y_0/x_0$
I	Price impact
Δy_{fr}	Token Y output from the front-run
Δy_{br}	Token Y input in the back-run
Δx_{br}	Token X output from the back-run
G_{JIT}	Gas cost of mint + burn operations for JIT
G_{br}	Gas cost of back-running swap
G_{sand}	Total gas paid by the sandwich attacker
m_{JIT}	Bribe or validator payment in JIT
m_{br}	Bribe or validator payment in back-run strategy
m_{sand}	Bribe or validator payment in sandwich attack

Table 1: Unified notation for MEV strategy modeling.

3.1 JIT profits

A JIT liquidity provider (LP) monitors the public Ethereum mempool and spots a pending swap of size S from X to Y . The LP constructs a single-bundle transaction with the following operations:

Mint. Just before the victim swap, the MEV searcher provides liquidity in an ultra-narrow tick range (typically, the smallest possible one dictated by the pool parameter *tickSpacing*). If the current tokens' reserves are x_0, y_0 , she provides αx_0 and αy_0 , with $\alpha > 0$. Hence, she owes a fraction $\frac{\alpha}{\alpha+1}$ of the range. The reserves are now

$$(x_0, y_0) \mapsto (x_1, y_1) = (x_0(1 + \alpha), y_0(1 + \alpha)) \quad (4)$$

Victim swap(s) ($X \rightarrow Y$). The victim swaps $S = x_0 \sigma$ tokens X for Δy_v tokens Y such that

$$x_1 y_1 = (x_1 + rS)(y_1 - \Delta y_v) \implies \Delta y_v = \frac{y_0(1 + \alpha)r\sigma}{1 + \alpha + r\sigma} \quad (5)$$

Therefore the reserves become

$$(x_1, y_1) \mapsto (x_2, y_2) = \left(x_0(1 + \alpha + r\sigma), \frac{y_0(1 + \alpha)^2}{1 + \alpha + r\sigma} \right) \quad (6)$$

Burn. Now, the MEV searcher removes her liquidity, burning a fraction $\frac{\alpha}{\alpha+1}$ of the new reserves (x_2, y_2) .

$$\Delta x_{\text{burn}} = \frac{\alpha}{1 + \alpha} x_0(1 + \alpha + r\sigma) \quad (7)$$

$$\Delta y_{\text{burn}} = \frac{\alpha}{1 + \alpha} \frac{y_0(1 + \alpha)^2}{1 + \alpha + r\sigma} \quad (8)$$

The difference between the burn amount and the mint amount is

$$\Delta x_{burn} - \alpha x_0 = \frac{\alpha}{\alpha + 1} x_0 r \sigma \quad (9)$$

$$\Delta y_{burn} - \alpha y_0 = -\frac{\alpha y_0 r \sigma}{1 + \alpha + r \sigma} \quad (10)$$

We will omit, for the moment, the hedging costs that the JIT LP faces when providing liquidity. Indeed, she takes the other part of the trade with respect to the victim, exposing herself to the token. Ideally, she would open an opposite position on another venue (usually, a CEX) to cover it ¹¹. Therefore, the LP's expected profit is bounded above by

$$\Pi_{JIT} = \underbrace{\frac{\alpha}{\alpha + 1} \sigma x_0 f}_{\text{fee earned}} + \underbrace{\frac{\alpha}{\alpha + 1} x_0 r \sigma}_{\text{LP change in token X}} - \underbrace{\frac{\alpha y_0 r \sigma}{1 + \alpha + r \sigma} \frac{x_0}{y_0}}_{\text{LP change in token Y valued at } P_0} \quad (11)$$

$$= \frac{\alpha}{\alpha + 1} x_0 \sigma - \frac{\alpha r \sigma}{1 + \alpha + r \sigma} x_0 \quad (12)$$

3.2 Back-running arbitrage option

The simplest alternative to JIT is to just arbitrage the price impact generated by the user transaction. A classic MEV back-runner would take a flash loan ¹² and lets the victim's swap push the pool price. Then she would trade in the opposite direction to harvest the transient spread.

In this case, the bundle is composed by just two operations

Victim's swap The victim swaps S tokens X in the pool (direction $X \rightarrow Y$). After the victim leg the reserves become

$$(x_0, y_0) \mapsto (x_1, y_1) = (x_0 + rS, y_0 - \Delta y_v) \quad (13)$$

where

$$\Delta y_v = y_0 \frac{r \sigma}{1 + r \sigma}. \quad (14)$$

If P_0 is the initial price and P_1 are, respectively, the initial and final prices in terms of the X token, the price impact I induced by this swap is

$$I = \frac{P_1}{P_0} - 1 = \frac{1}{(1 + r \sigma)^2} - 1 \quad (15)$$

Back-run arbitrage trade The arbitrageur must choose the optimal amount Δy_{br} to trade in the opposite direction (swap $Y \rightarrow X$) to maximize her profits Π_{br} . In general the profit function can be written as the difference between what she obtain from the arbitrage swap, indicated with Δx_{br} , and what she has paid. Therefore, evaluating everything in token X , we have

$$\Pi_{br} = \Delta x_{br} - \frac{\Delta y_{br}}{P_0} \quad (16)$$

where P_0 is the initial price. Now we have to workout the value of Δx_{br} as a function of Δy_{br} . This can be simply obtained from the constant product rule

$$(x_1 - \Delta x_{br})(y_1 + r \Delta y_{br}) = x_1 y_1 \implies \Delta x_{br} = \frac{x_1 r \Delta y_{br}}{y_1 + r \Delta y_{br}} \quad (17)$$

Using (16), we obtain

$$\Pi_{br} = \frac{x_1 r \Delta y_{br}}{y_1 + r \Delta y_{br}} - \frac{\Delta y_{br}}{P_0} \quad (18)$$

¹¹Therefore, the profit equation must be seen as an upper bound.

¹²We assume that the cost of the flash loan is zero.

Setting the derivative of Π_{br} to zero and solving for Δy_{br} , we obtain that the optimal back-run size is

$$\Delta y_{br}^* = \frac{y_0}{r} \left[\sqrt{r} - \frac{1}{1+r\sigma} \right] \quad (19)$$

that is greater than zero when

$$\sigma \geq \frac{1/\sqrt{r} - 1}{r} \simeq \frac{f}{2} \quad (20)$$

Or, equivalently, when $|I| > f$.

The price after the optimal trade is

$$P_2 = \frac{y_2}{x_2} = rP_0 \quad (21)$$

The arbitrageur trades until the final price equals the start price after deducting the fee; therefore, a positive fee always leaves an un-reverted spread. In the case of zero fee, the price is totally reverted, bringing back the reserves to their initial state. Plugging Δy_{br}^* into (18) we obtain the maximum profit achievable

$$\Pi_{br}^*(\sigma) = x_0 \frac{(\sqrt{r}(1+r\sigma) - 1)^2}{r(1+r\sigma)} \quad (22)$$

3.3 Sandwich attack option

In this section we show that the sandwich option increases the magnitude of the victim's price impact that makes the alternative an economically viable strategy. The results are similar to what we can found in [?] and [?], although here we make explicit different variables.

We analyse a “self-funded” sandwich on a constant-product AMM: the MEV searcher first swaps $X \rightarrow Y$ (front-run) and immediately re-uses the received Y to swap back $Y \rightarrow X$ (back-run).

The AMM pool starts with reserves x_0, y_0 .

Front-run swap ($X \rightarrow Y$). After the front-run swap of size s , they become $x_0 + s, y_0 - \Delta y_{fr}$, where

$$\Delta y_{fr} = y_0 \frac{r\varepsilon}{1+r\varepsilon}. \quad (23)$$

Victim swap(s) ($X \rightarrow Y$). Now the searcher lets the victim swap. Therefore the new reserves become $x_0 + rs + rS, y_0 - \Delta y_{fr} - \Delta y_v$, where

$$\Delta y_v = \frac{\left(\frac{y_0}{1+r\varepsilon} \right) r\sigma}{1+r\varepsilon+r\sigma} \quad (24)$$

If γ is the slippage tolerance, the victim's swap succeeds if and only if Δy_v is at least equal to $(1-\gamma)$ the quantity that the victim would have received without being front-run Δy_{base} , equal to

$$\Delta y_{base} = y_0 \frac{r\sigma}{1+r\sigma}. \quad (25)$$

So we have that

$$\Delta y_v \geq (1-\gamma)\Delta y_{base} \quad (26)$$

that leads to a quadratic inequality for ε whose lower root is negative. Finally, we have that

$$0 \leq \varepsilon \leq \varepsilon_{\max}(\sigma, \gamma, r) = \frac{1}{r} \left[\frac{-r\sigma + \sqrt{r^2\sigma^2 + \frac{4(1+r\sigma)}{1-\gamma}}}{2} - 1 \right] \quad (27)$$

with the edge cases $\gamma = 0 \implies \varepsilon_{\max} = 0$ and $\gamma = 1 \implies \varepsilon_{\max} \rightarrow \infty$. This is a constraint that the optimal ε must satisfy. Figure 4 shows the behavior of $\varepsilon_{\max}$ for $r = 0.997$ and $\gamma = 0.01$

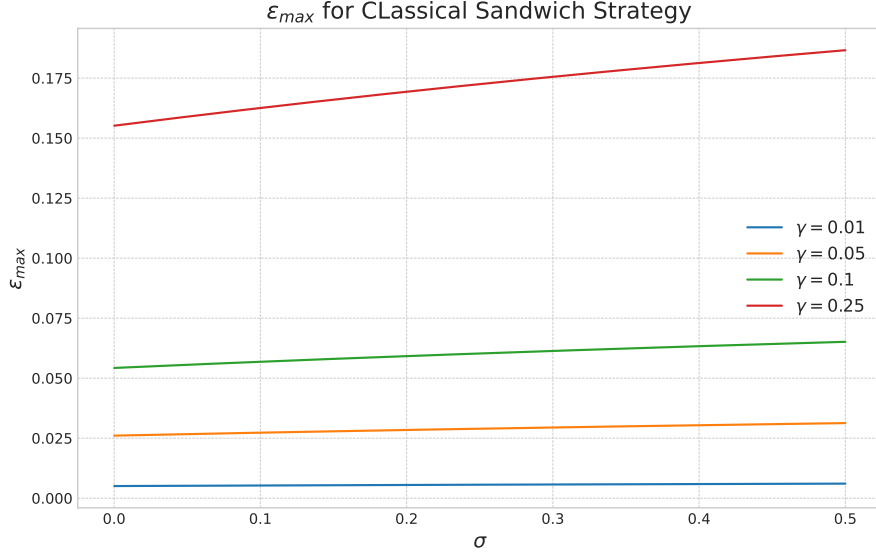


Figure 4: Caption

Back-run swap (Y→X). Finally, Δy_{fr} are traded back in the back-run swap from Y to X . The amount that the searcher gets back is

$$\Delta x_{br} = x_0 \frac{r^2 \varepsilon (1 + r\varepsilon + r\sigma)^2}{(1 + r\varepsilon) + r^2 \varepsilon (1 + r\varepsilon + r\sigma)}. \quad (28)$$

Not considering for the moment gas and bribe costs, the searcher's profit is the difference between the input amount in the front-run swap and the output amount in the back-run swap

$$\Pi_{sand}(\varepsilon, \sigma) = \Delta x_{br} - s = x_0 \left[\frac{r^2 \varepsilon (1 + r\varepsilon + r\sigma)^2}{(1 + r\varepsilon) + r^2 \varepsilon (1 + r\varepsilon + r\sigma)} - \varepsilon \right], \quad (29)$$

$$\pi_{sand}(\varepsilon, \sigma) = \frac{\Pi}{x_0} \quad (30)$$

We can obtain the value of σ that turn on the profits:

$$\pi_{sand}(\varepsilon, \sigma) > 0 \quad \text{when} \quad \sigma > \sigma_{min} = \frac{1}{r} \left(\frac{\varepsilon + \sqrt{\varepsilon^2 + \frac{4(1+r\varepsilon)}{r^2}}}{2} - 1 - r\varepsilon \right) \underset{\varepsilon \rightarrow 0^+}{\approx} f \quad (31)$$

Now we want to obtain the optimal value to trade in the front-run swap that maximizes the profit. In other words, we want to solve the following problem

$$\begin{aligned} & \text{maximize} \quad \pi_{sand}(\varepsilon, \sigma) \\ & \text{subject to} \quad \varepsilon \leq \varepsilon_{\max}(\sigma, \gamma, r) \end{aligned} \quad (32)$$

The first-order condition leads to a quartic equation whose solution can be found numerically.

The constraint given by the slippage requires that the actual maximum ε^* be

$$\varepsilon^* = \min \{ \varepsilon_+, \varepsilon_{\max}(\sigma, \gamma, r) \} \quad (33)$$

If we substitute this value in the profit expression, we get

$$\pi_{sand}^*(\varepsilon, \sigma) = \frac{r^2 \varepsilon^* (1 + r\varepsilon^* + r\sigma)^2}{(1 + r\varepsilon^*) + r^2 \varepsilon^* (1 + r\varepsilon^* + r\sigma)} - \varepsilon^* \quad (34)$$

Define the gas spread and the gas spread per reserve.

$$\delta' := G_{JIT} - G_{sand}, \quad \phi' := \frac{\delta'}{x_0} \quad (35)$$

The two bribe ceilings are

$$\begin{aligned} m_{\max}^{\text{JIT}} &= Sf - G_{\text{JIT}} = x_0(\sigma f - \phi'), \\ m_{\max}^{\text{sand}} &= x_0 \pi^*. \end{aligned}$$

The builder includes the JIT bundle when its bribe ceiling exceeds the sandwich one:

$$x_0(\sigma f - \phi') \geq x_0 \pi^* \iff F(\sigma) := \sigma f - \phi' - \pi^* \geq 0. \quad (36)$$

Because $F(0) = -\phi' < 0$ (if JIT has any gas handicap) and $F(\sigma) \rightarrow \infty$ as $\sigma \rightarrow \infty$, there is (under regular fee-tier ranges) a unique positive root σ_{sand}^* that solves $F(\sigma) = 0$. Therefore, we have that the JIT strategy wins against the sandwich attack alternative when

$$\boxed{\sigma \geq \sigma_{\text{sand}}^*(f, \phi')} \iff \boxed{S \geq S_{\text{sand}}^* := \sigma_{\text{sand}}^* x_0}. \quad (37)$$

Figure 9 represents the relationship between the front-run size ε and the sandwich profit $\pi(\varepsilon)$. As we can see, the profit reaches a maximum and then starts to decrease afterward. The decreasing can also lead to negative profits, especially for low liquidity pools. However, the maximum is never actually reached by the searcher. Also in this case, the slippage tolerance does not let the attacker to extract the most of the rent.

3.4 Mixed (Sandwich + JIT) attack

We analyze a bundle that combines a self-funded sandwich with JIT liquidity provision. The sequence consists of a front-run swap, a front-run mint, a set of victim swaps, a back-run burn, and a back-run swap. The goal is to profit both from the sandwich's price impact and the JIT fee capture. Because the JIT deepens the book during the victim leg, the victim's min-out constraint is easier to satisfy, which allows a larger feasible front-run size ε . To the best of our knowledge, there is currently no study in the literature that accounts for this strategy. Furthermore, we were not able to find any particular information even on the web, even if data clearly shows the occurrence of this pattern, as highlighted in [?]

Front-run swap (X→Y). The attacker swaps $s = \varepsilon x_0$ (user input), so the effective input is rs . With constant product and fee on input,

$$\Delta y_{\text{fr}} = y_0 \frac{r\varepsilon}{1 + r\varepsilon}, \quad (x_0, y_0) \mapsto (x_1, y_1) = (x_0(1 + r\varepsilon), y_0/(1 + r\varepsilon)). \quad (38)$$

Front-run JIT mint. To avoid changing price, it is convenient to parametrize the added reserves with respect to the current ones. If α of the *post-FR* reserves is added, the state scales as

$$(x_1, y_1) \mapsto (x_2, y_2) = (x_1(1 + \alpha), y_1(1 + \alpha)) = \left(x_0(1 + r\varepsilon)(1 + \alpha), \frac{y_0(1 + \alpha)}{1 + r\varepsilon}\right). \quad (39)$$

Therefore the attacker owns a fraction $\frac{\alpha}{\alpha+1}$ of the active range.

Victim swap(s) (X→Y). Size $S = \sigma x_0$; effective input rS . From (39), the victim's Y out is

$$\Delta y_v = \left(\frac{y_0}{1 + r\varepsilon}\right) \cdot \frac{(1 + \alpha) r\sigma}{(1 + r\varepsilon)(1 + \alpha) + r\sigma}, \quad (40)$$

and the state becomes (still with JIT present)

$$(x_3, y_3) = \left(x_0((1 + r\varepsilon)(1 + \alpha) + r\sigma), \frac{y_0(1 + \alpha)^2}{(1 + r\varepsilon)(1 + \alpha) + r\sigma}\right). \quad (41)$$

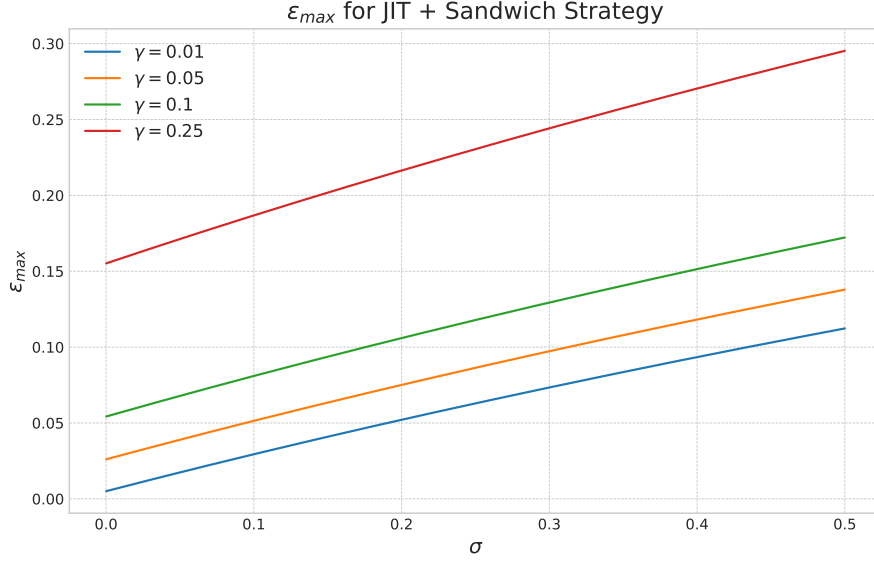


Figure 5: Maximum normalized front run amount ϵ_{max} vs normalized victim's swap amount σ for different values of the slippage tolerance γ . The value of α is set to 1 (i.e. LP owns half of the active tick range).

Slippage feasibility is the same as in §3.3 but in this case the enhanced liquidity allows for a bigger ϵ_{max} (see Figure 5):

$$\Delta y_v \geq (1 - \gamma) \Delta y_{base}, \quad \Delta y_{base} = y_0 \frac{r\sigma}{1 + r\sigma}, \quad \implies \quad \epsilon \leq \epsilon_{max}(\sigma, \gamma, r, \alpha). \quad (42)$$

where

$$\epsilon_{max}(\sigma, \gamma, r, \alpha) = \frac{1}{r} \left[\frac{-r\sigma + \sqrt{(r\sigma)^2 + \frac{4(1+\alpha)^2(1+r\sigma)}{1-\gamma}}}{2(1+\alpha)} - 1 \right] \quad (43)$$

Back-run burn (JIT). Immediately after the victim, the JIT LP is burned. Since the position was proportional, burning removes a fraction $\alpha/(1+\alpha)$ of both reserves, so the base-pool state is

$$(x_4, y_4) = \frac{1}{1+\alpha} (x_3, y_3) = \left(x_0 \left((1+r\epsilon) + \frac{r\sigma}{1+\alpha} \right), \frac{y_0(1+\alpha)}{(1+r\epsilon)(1+\alpha) + r\sigma} \right). \quad (44)$$

After the burn, the attacker ends up with

$$\Delta x_{burn} = \frac{\alpha}{\alpha+1} x_3 = \frac{x_0 \alpha}{\alpha+1} \quad (45)$$

$\alpha x_0(1+r\epsilon) + \frac{\alpha}{\alpha+1} r\sigma x_0$ tokens X and $\alpha y_0 - \frac{\alpha}{\alpha+1} \Delta y_v$ of tokens Y.

Back-run swap (Y→X). The attacker now has to swap in the opposite direction than victims' swaps.

$$x_4 y_4 = (x_4 - \Delta x_{br})(y_4 + r \Delta y_{br}) \quad (46)$$

We can derive the expression of Δx_{br} with respect to x_0 . Let $\theta = \frac{\Delta y_{br}}{y_0}$

$$\Delta x_{br} = x_0 \left\{ \frac{[(1+r\epsilon)(1+\alpha) + r\sigma]^2 r\theta}{(1+\alpha)[(1+\alpha) + ((1+\alpha)(1+r\epsilon) + r\sigma)r\theta]} \right\} \quad (47)$$

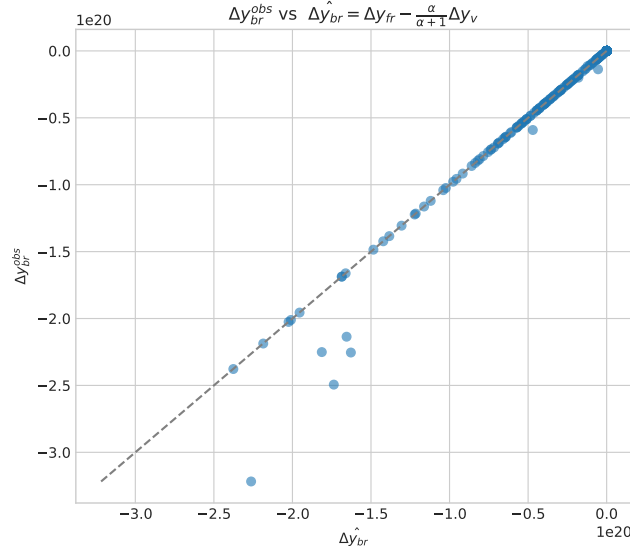


Figure 6: Relation between the back run swap amount observed on-chain Δy_{br}^{obs} and the theoretical one Δy_{br} based on the self funding condition.

Profit. The profit the attacker obtains is composed of the difference between the back-run output amount and the front-run input amount (the same as in the case of the classical sandwich) plus the fees collected by the LP position plus the difference between the tokens' amounts burnt and minted

$$\Pi_{sand+jit}(\varepsilon, \sigma, \alpha) = \underbrace{(\Delta x_{br} - \varepsilon x_0)}_{\text{Sandwich}} + \underbrace{\sigma x_0 f \frac{\alpha}{\alpha+1}}_{\text{LP fees}} + \underbrace{x_0 \alpha r \sigma \left[\frac{1}{1+\alpha} - \frac{1}{(1+r\varepsilon)((1+\alpha)(1+r\varepsilon)+r\sigma)} \right]}_{\text{LP inventory}} \quad (48)$$

Therefore, normalizing the profit with respect to x_0 and substituting the full expression for Δx_{br} , we obtain

$$\pi_{sand+jit}(\varepsilon, \alpha, \sigma) = \left\{ \frac{[(1+r\varepsilon)(1+\alpha)+r\sigma]^2 r\theta}{(1+\alpha)[(1+\alpha)+((1+\alpha)(1+r\varepsilon)+r\sigma)r\theta]} - \varepsilon \right\} + \frac{\sigma f \alpha}{\alpha+1} + \alpha r \sigma \left[\frac{1}{1+\alpha} - \frac{1}{(1+r\varepsilon)((1+\alpha)(1+r\varepsilon)+r\sigma)} \right] \quad (49)$$

In general, we have two options for θy_0 : (i) keep the self funding framework as in the classical sandwich attack or (ii) let the back run amount to be a new free parameter. Currently we did not study the second option since empirically we almost always find the self funded option, as we can see from Figure 6.

Differently from the case of the self funded sandwich attack, in this case the back run amount is not equal to the front run amount, because the LP position has already hedged part of the attacker exposure obtained after the front run. Specifically, during the victims' swap, the attacker takes the opposite side of the trades, giving $\frac{\alpha}{(1+\alpha)}\Delta y_v$ tokens Y and receiving $\frac{\alpha}{(1+\alpha)}rS$ of tokens X. Therefore, she has only to back run the residual $\theta_{self} = \frac{1}{y_0} \left(\Delta y_{fr} - \frac{\alpha}{1+\alpha} \Delta y_v \right)$. Empirically it is actually what we see.

In this case, the LP inventory change in tokens Y is fed as a minus component into the back run amount. The profit is therefore

$$\Pi_{sand+jit}(\varepsilon, \alpha, \sigma) = \underbrace{(\theta_{self} y_0 - \varepsilon x_0)}_{\text{Sandwich Profit}} + \underbrace{\frac{\alpha \sigma x_0 f}{\alpha+1}}_{\text{LP fees}} + \underbrace{\frac{x_0 \alpha r \sigma}{\alpha+1}}_{\text{LP inventory change in X}} \quad (50)$$

The full expression is

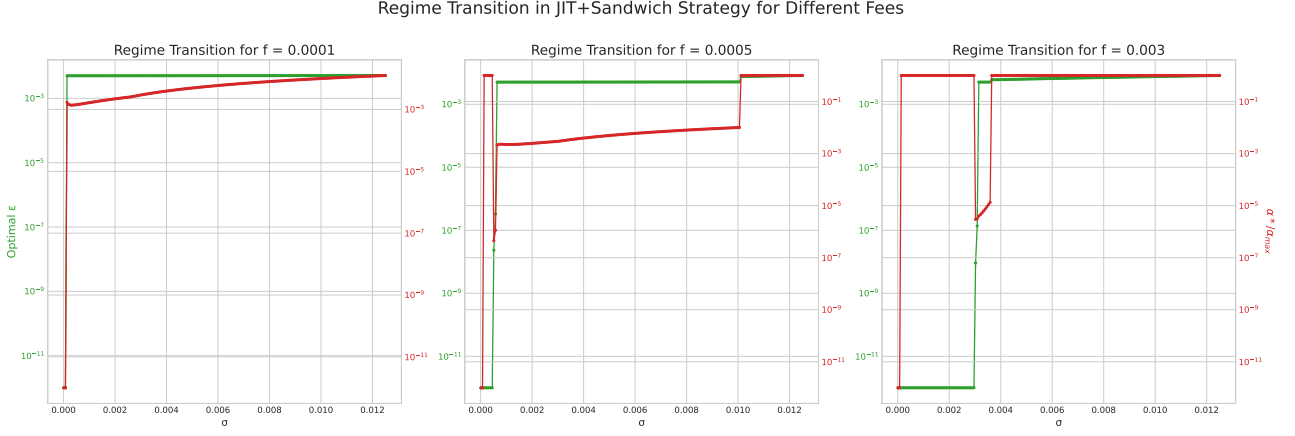


Figure 7: Optimal values for ε and α with respect to σ for different values of the fee tier.

$$\pi_{sand+jit}(\varepsilon, \alpha, \sigma) = \frac{[(1+r\varepsilon)(1+\alpha) + r\sigma]^2 r \theta_{self}}{(1+\alpha)[(1+\alpha) + ((1+\alpha)(1+r\varepsilon) + r\sigma)r\theta_{self}]} - \varepsilon + \frac{\alpha\sigma}{\alpha+1} \quad (51)$$

where

$$\theta_{self} = \frac{r}{1+r\varepsilon} \left[\varepsilon - \frac{\alpha\sigma}{(1+r\varepsilon)(1+\alpha) + r\sigma} \right] \quad (52)$$

When $\theta_{self} \leq 0$ the (normalized) fees earned by the LP is bigger than ε , flipping the sign of the back run swap. Therefore, to not incur in a loss, the MEV searcher chooses to perform a pure JIT.

The maximization problem is

$$\max_{\varepsilon, \alpha} \pi_{sand+jit} \quad \text{subject to} \quad \begin{cases} 0 \leq \varepsilon \leq \varepsilon_{\max} \\ \alpha \geq 0 \\ \theta_{self} > 0 \end{cases} \quad (53)$$

The results have been obtained numerically searching for $\varepsilon^* \in [\varepsilon_{\min}, \infty]$ and $\alpha \in [0, \alpha_{\max}]$ with $\varepsilon_{\min} = 10^{-9}$ and $\alpha_{\max} = 1$. Actually, the optimizer always pushes $\alpha^* \rightarrow \alpha_{\max}$, whatever its value is. Figure 7 shows ε^* and α^* as a function of the normalized victim's input size σ for $f = 0.01\%, 0.05\%, 0.3\%$. Except in the case of the 0.01%, we can see that there is a quite sharp transition between two different regimes:

1. Pure JIT, where ε is pushed to $\varepsilon_{\min}$ and α to $\alpha_{\max}$
2. Combination of JIT and Sandwich attack with

Figure 3.4 shows the dynamics of the four ceilings (profits) for the three MEV strategies discussed so far for different fee tiers. One can be surprised by the fact that a classic sandwich attack is less profitable than an arbitrage trade after a certain value of σ . This is due to how we treated the sandwich attack strategy. Indeed, we set up a self funded strategy, in which the back run amount is equal to the front run size. Due to the presence of the slippage tolerance, the maximum front-run size $\varepsilon_{\max}$ is limited. Therefore, also the maximum profit achievable will be affected by this constraint. On the other hand, we do not have any limit on the arbitrage back-run that depends only on the victim swapped size σ . Anyway, combining a classical sandwich with a JIT, allows for a greater front run size and a flow of fee towards the searcher. This makes the mixed strategy the most profitable one for $f = 0.3\%$. For lower ones, the difference between the classical sandwich and the mixed strategy becomes negligible. As we can see from the graphs, the transition between a pure JIT and a mixed strategy happens at $\sigma = f$.

Comparison of MEV Strategy Profits ($\gamma=0.01$)

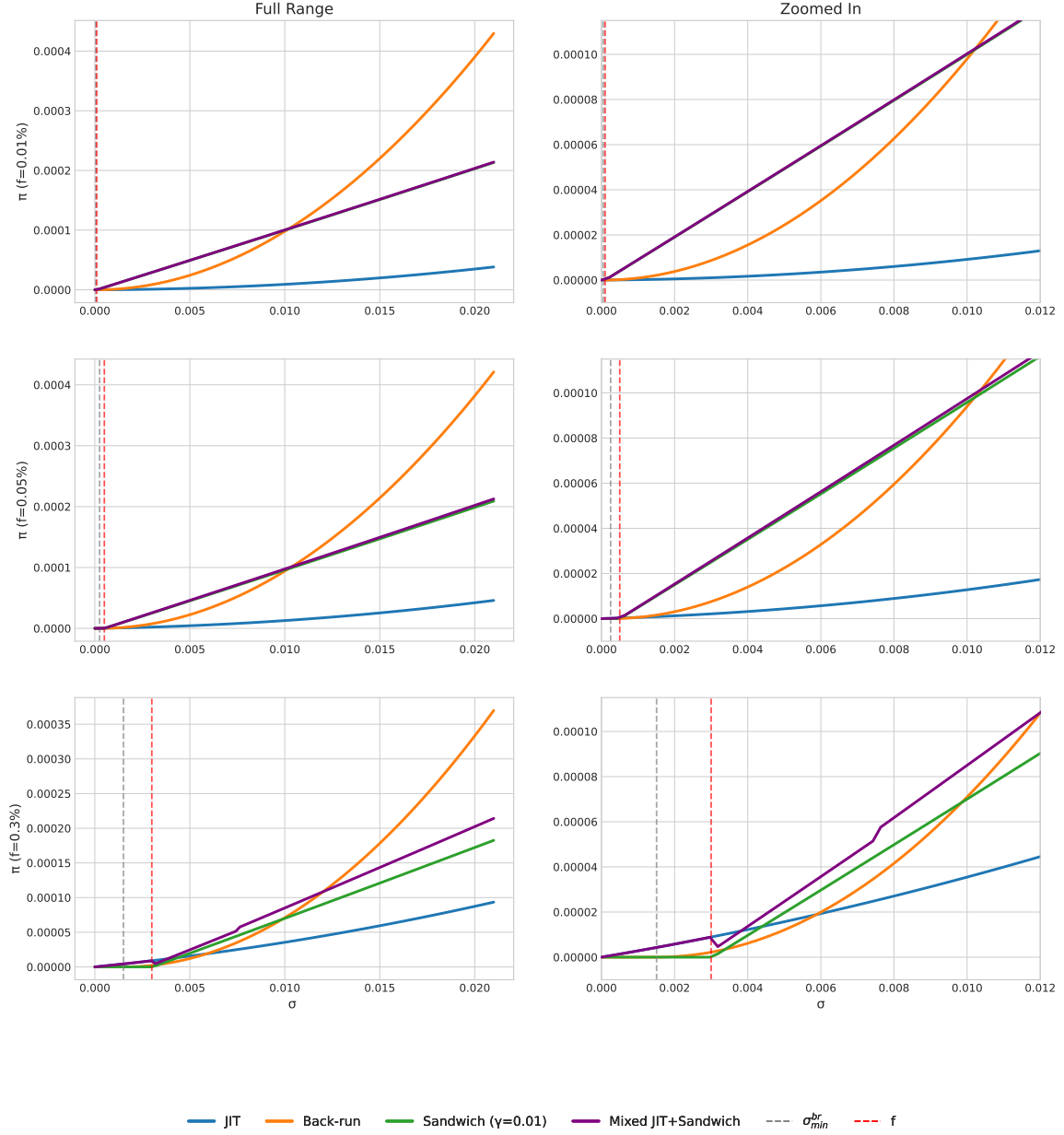


Figure 8: Visualization of the profits of the different MEV strategies for $f \in [0.01\%, 0.05\%, 0.3\%]$ and slippage $\gamma = 1\%$. The vertical gray dashed line corresponds to the minimum value of σ that induces a price change greater than the fee tier, while the red dashed line highlights the pool's fee rate. The gas fees are set to zero.

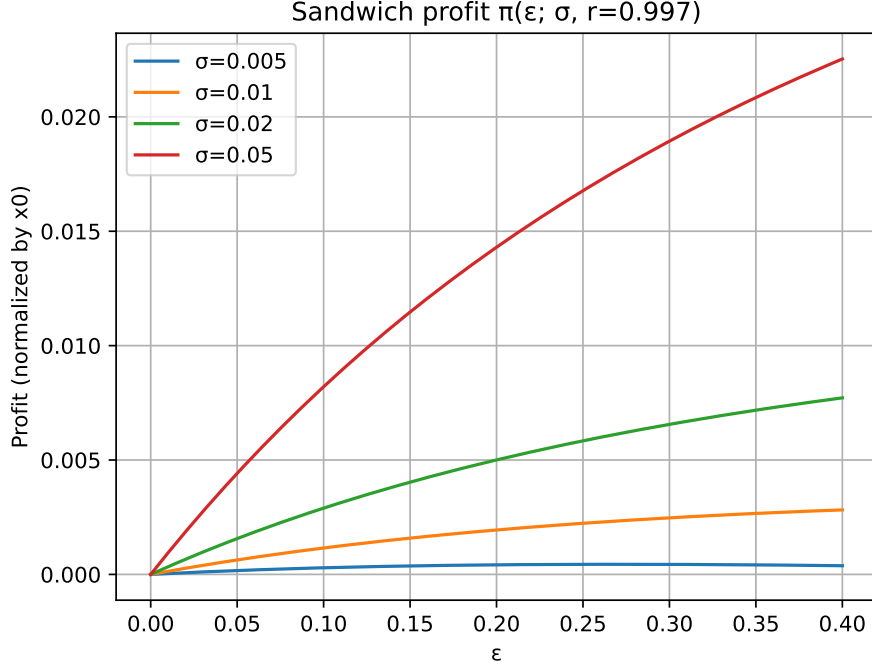


Figure 9: Sandwich profits vs ε . The vertical dashed lines correspond to the maximum value of the front run size $\varepsilon_{max}(\gamma)$ for different value of the slippage tolerance γ . Fee tier 0.05%, $x_0 = 1$.

4 Empirical profits

Currently I have collected MEV patterns discussed in the previous sections for the 2023 and for the USDC-WETH 0.05% pool. The goal is to check the validity of the relationship obtained in the previous section regarding the MEV strategy profits.

The heuristic used to collect the arbitrage swaps is inherently noisy: a swap that changed the price for a quantity greater than the fee tier is not necessarily followed by an arbitrage swap. In other words, even if there is a swap in the opposite direction, it can be a coincidence. Therefore, I applied a stronger heuristic, collecting only the swaps that pushed the price back to 1% of the target rP_0 .

Profits are evaluated based on actual on-chain data as the difference between what the searcher paid and what was obtained from the strategy. There is therefore no bias due to the theory discussed above, except for the condition imposed while collecting back run arbitrages.

Finally, no gas costs have been considered.

The JIT and classical sandwich attacks profits seem to follow the behavior obtained in the theoretical sections. However, for sandwich profits we found

- A lot of negative profits. Almost all of them are related to $\sigma < \sigma_{min}$
- A lot of back run arbitrages with almost zero profits

As shown in Figure 10, there are some discrepancies between theory and empirical evidences. For instance, the onset of positive profits for both the sandwich attacks/mixed strategy and the back run arbitrages happens for σ values a bit greater than the predicted ones. Possible sources of this discrepancy can be (i) not optimality in strategy settings or, more probably, (ii) cross-tick effects that we did not account for

A Tick-aware rules for Uniswap v3 execution

State and notation. Let $S = \sqrt{P}$ denote the square-root price (X in Y) and L the *active* liquidity of the current tick whose boundaries are (S_ℓ, S_u) with $S_\ell < S < S_u$. The pool charges a fee-on-input f ; write $r = 1 - f$. For intuition within a tick, define the virtual reserves $\tilde{x} = L/S$ and $\tilde{y} = LS$ (so $\tilde{x}\tilde{y} = L^2$). Only *swaps* move price; *mints/burns* are price-neutral.

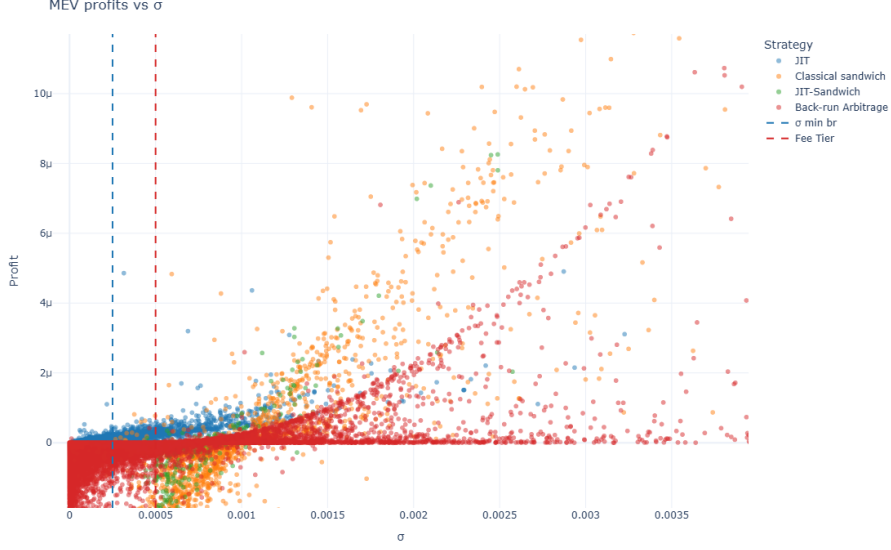


Figure 10: Caption

A.1 Swaps within a tick (price update and amounts)

X→Y (consume X, output Y). For effective input $x_{\text{eff}} = r x_{\text{in}}$:

$$S' = \frac{1}{S^{-1} + x_{\text{eff}}/L}, \quad \Delta y = L(S' - S), \quad \text{fees in X} = f x_{\text{in}}. \quad (54)$$

Y→X (consume Y, output X). For effective input $y_{\text{eff}} = r y_{\text{in}}$:

$$S' = S + \frac{y_{\text{eff}}}{L}, \quad \Delta x = L(S^{-1} - S'^{-1}), \quad \text{fees in Y} = f y_{\text{in}}. \quad (55)$$

Interpretation. Within a tick these are exactly the constant-product updates on $(\tilde{x}, \tilde{y})$ using the *effective* input r (fees are withheld from input and added to the pool in the input token). Signs $\Delta x, \Delta y$ above are pool-side deltas (amounts received by the trader have opposite sign).

A.2 Tick boundaries (capacity and traversal)

Capacity to the lower boundary S_ℓ for a *downward* move (X→Y):

$$x_{\text{max}}^{(\text{eff})} = L \left(\frac{1}{S_\ell} - \frac{1}{S} \right), \quad x_{\text{max}} = \frac{x_{\text{max}}^{(\text{eff})}}{r}. \quad (56)$$

Capacity to the upper boundary S_u for an *upward* move (Y→X):

$$y_{\text{max}}^{(\text{eff})} = L(S_u - S), \quad y_{\text{max}} = \frac{y_{\text{max}}^{(\text{eff})}}{r}. \quad (57)$$

Boundary rule. When a boundary is reached, update the active liquidity

$$L \leftarrow L \pm \Delta L_{\text{tick}}, \quad (58)$$

where ΔL_{tick} is the net liquidity change encoded at that boundary; then advance to the next active tick (S_ℓ, S_u) and continue. (Only the *active* tick's L affects price moves.)

A.3 Liquidity changes (mints and burns)

General mint (range $[S_a, S_b]$ with $S_a < S_b$). Adding ΔL does *not* move price. The deposited principal amounts at current S are

$$(\Delta x, \Delta y) = \begin{cases} \left(\Delta L \left(\frac{1}{S_a} - \frac{1}{S_b} \right), 0 \right), & S \leq S_a, \\ \left(\Delta L \left(\frac{1}{S} - \frac{1}{S_b} \right), \Delta L (S - S_a) \right), & S_a < S < S_b, \\ \left(0, \Delta L (S_b - S_a) \right), & S \geq S_b. \end{cases} \quad (59)$$

If $S \in [S_a, S_b)$, the position is *active* and the active liquidity becomes $L \leftarrow L + \Delta L$. If $S \notin [S_a, S_b)$, the deposit is *inactive now* (no change to active L) and becomes active only when price re-enters the range.

JIT mint (ultra-narrow around the current tick). Take $[S_a, S_b]$ to coincide with the current tick; set $\alpha = \Delta L / L_{\text{base}}$. Then, while price stays in the tick,

$$L \leftarrow (1 + \alpha) L_{\text{base}}, \quad (\tilde{x}, \tilde{y}) \text{ scale by } (1 + \alpha). \quad (60)$$

This is the formal justification for the “state scales by $1 + \alpha$ ” step used in Section 3. If price leaves the tick before a burn, the JIT position becomes *inactive* (it no longer contributes to active L) until price re-enters or the position is burned.

General burn (range $[S_a, S_b]$). Removing ΔL does *not* move price. The withdrawn principal at current S is

$$(\Delta x, \Delta y) = \begin{cases} \left(\Delta L \left(\frac{1}{S_a} - \frac{1}{S_b} \right), 0 \right), & S \leq S_a, \\ \left(\Delta L \left(\frac{1}{S} - \frac{1}{S_b} \right), \Delta L (S - S_a) \right), & S_a < S < S_b, \\ \left(0, \Delta L (S_b - S_a) \right), & S \geq S_b. \end{cases} \quad (61)$$

If the position is *active* at burn time, update active liquidity $L \leftarrow L - \Delta L$. If the position is *inactive*, burning does not change active L (you still withdraw principal computed at current S per the cases above).

Fee accruals for mints/burns. Fees are collected *pro-rata to active liquidity and only while the position is active*. In a swap that consumes input q with fee f , the pool retains $f q$ of the *input* token; a position active during that swap accrues the share $\frac{\Delta L}{L_{\text{active}}} \times f q$ (up to rounding) into its uncollected fees. Burning (or an explicit **collect**) realizes the position’s uncollected fees in their respective tokens *in addition to* the principal amounts above. For a JIT mint active only for the victim swap, the fee share during that swap is $\frac{\alpha}{1 + \alpha}$ of the victim’s fee in the input token (again subject to rounding and any protocol fee).